Báo cáo Chương 2 — Memory Safety & Static C Checker

Mục lục

1. Mã nguồn được phân tích **2**

2. Kết quả phân tích **5**

1. Mã nguồn được phân tích

Mã nguồn C

```
#include "../include/product.h"
#include "../include/validation.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <stdint.h>

/* Khởi tạo danh sách sản phẩm động */
int product_list_init(ProductList *list) {
if (list == NULL) {
return STATUS_ERR_NULL_PTR;
}

list->items = (Product *)malloc(INITIAL_CAPACITY * sizeof(Product));
if (list->items == NULL) {
list->count = 0;
list->capacity = 0;
return STATUS_ERR_MEMORY;
}

list->count = 0;
list->capacity = INITIAL_CAPACITY;
return STATUS_SUCCESS;
}

/* Giải phóng bộ nhớ động của danh sách sản phẩm */
void product_list_free(ProductList *list) {
if (list != NULL && list->items != NULL) {
free(list->items);
list->items = NULL;
list->count = 0;
list->capacity = 0;
}
}

/* Hàm hỗ trợ sao chép chuỗi an toàn, luôn đảm bảo ký tự kết thúc '\0' */
static void safe_strcpy(char *dest, const char *src, size_t dest_size) {
if (dest == NULL || dest_size == 0) return;
if (src == NULL) {
dest[0] = '\0';
return;
}
snprintf(dest, dest_size, "%s", src);
}

/* Hàm hỗ trợ so sánh chuỗi không phân biệt hoa thường */
static int contains_ignore_case(const char *haystack, const char *needle) {
if (haystack == NULL || needle == NULL) return 0;
if (needle[0] == '\0') return 1;

char h_lower[MAX_NAME_LEN];
char n_lower[MAX_NAME_LEN];

size_t i = 0;
for (; haystack[i] != '\0' && i < sizeof(h_lower) - 1; ++i) {
h_lower[i] = (char)tolower((unsigned char)haystack[i]);
}
h_lower[i] = '\0';

size_t j = 0;
for (; needle[j] != '\0' && j < sizeof(n_lower) - 1; ++j) {
n_lower[j] = (char)tolower((unsigned char)needle[j]);
}
n_lower[j] = '\0';
```

```
return strstr(h_lower, n_lower) != NULL;
}

/* TASK 3: Cài đặt chức năng Thêm sản phẩm vào danh sách động */
int add_product(ProductList *list, const Product *prod) {
if (list == NULL || prod == NULL) {
return STATUS_ERR_NULL_PTR;
}

/* 1. Kiểm tra tính hợp lệ của mã sản phẩm (ID) bằng Validation Module */
int val_id = validate_product_id(prod->id);
if (val_id != STATUS_SUCCESS) {
return val_id;
}

/* 2. Kiểm tra tính hợp lệ của tên sản phẩm bằng Validation Module */
int val_name = validate_product_name(prod->name);
if (val_name != STATUS_SUCCESS) {
return val_name;
}

/* 3. Kiểm tra tính hợp lệ của loại sản phẩm bằng Validation Module */
int val_cat = validate_category(prod->category);
if (val_cat != STATUS_SUCCESS) {
return val_cat;
}

/* 4. Kiểm tra tính hợp lệ của đơn vị tính bằng Validation Module */
int val_unit = validate_unit(prod->unit);
if (val_unit != STATUS_SUCCESS) {
return val_unit;
}

/* 5. Kiểm tra tính hợp lệ của số lượng (>= 0) bằng Validation Module */
int val_qty = validate_quantity(prod->quantity);
if (val_qty != STATUS_SUCCESS) {
return val_qty;
}

/* 6. Kiểm tra tính hợp lệ của đơn giá (>= 0) bằng Validation Module */
int val_price = validate_price(prod->price);
if (val_price != STATUS_SUCCESS) {
return val_price;
}

/* 7. Kiểm tra trùng lặp mã sản phẩm (Duplicate ID) */
if (find_product_by_id(list, prod->id) != NULL) {
return STATUS_ERR_DUPLICATE_ID;
}

/* 8. Mở rộng bộ nhớ mảng động khi đầy (Baseline v0 - [RISK-02]: Chưa có guard chống tràn số nguyên size_t) */
if (list->count >= list->capacity) {
size_t new_cap = (list->capacity == 0) ? INITIAL_CAPACITY : (list->capacity * 2);
Product *new_items = (Product *)realloc(list->items, new_cap * sizeof(Product));
if (new_items == NULL) {
return STATUS_ERR_MEMORY; /* Giữ nguyên list->items cũ, tránh rò rỉ bộ nhớ */
}
list->items = new_items;
list->capacity = new_cap;
}

/* 9. Sao chép thông tin sản phẩm mới vào mảng (bao gồm unit) */
Product *target = &list->items[list->count];
safe_strcpy(target->id, prod->id, sizeof(target->id));
safe_strcpy(target->name, prod->name, sizeof(target->name));
safe_strcpy(target->category, prod->category, sizeof(target->category));
safe_strcpy(target->unit, prod->unit, sizeof(target->unit));
target->quantity = prod->quantity;
target->price = prod->price;
```

```
list->count++;
return STATUS_SUCCESS;
}

/* TASK 4: Sửa thông tin sản phẩm theo mã ID */
int update_product(ProductList *list, const char *id, const char *new_name, const char *new_category, const char
*new_unit, int new_quantity, double new_price) {
if (list == NULL || id == NULL) {
return STATUS_ERR_NULL_PTR;
}

/* Tìm vị trí sản phẩm theo ID */
Product *target = find_product_by_id(list, id);
if (target == NULL) {
return STATUS_ERR_NOT_FOUND;
}

/* 1. Kiểm tra và cập nhật tên mới (nếu new_name != NULL) */
if (new_name != NULL) {
int val_name = validate_product_name(new_name);
if (val_name != STATUS_SUCCESS) {
return val_name;
}
safe_strcpy(target->name, new_name, sizeof(target->name));
}

/* 2. Kiểm tra và cập nhật loại sản phẩm mới (nếu new_category != NULL) */
if (new_category != NULL) {
int val_cat = validate_category(new_category);
if (val_cat != STATUS_SUCCESS) {
return val_cat;
}
safe_strcpy(target->category, new_category, sizeof(target->category));
}

/* 3. Kiểm tra và cập nhật đơn vị tính mới (nếu new_unit != NULL) */
if (new_unit != NULL) {
int val_unit = validate_unit(new_unit);
if (val_unit != STATUS_SUCCESS) {
return val_unit;
}
safe_strcpy(target->unit, new_unit, sizeof(target->unit));
}

/* 4. Kiểm tra và cập nhật số lượng mới (nếu new_quantity != -1) */
if (new_quantity != -1) {
int val_qty = validate_quantity(new_quantity);
if (val_qty != STATUS_SUCCESS) {
return val_qty;
}
target->quantity = new_quantity;
}

/* 5. Kiểm tra và cập nhật đơn giá mới (nếu new_price != -1.0) */
if (new_price >= 0.0) {
int val_price = validate_price(new_price);
if (val_price != STATUS_SUCCESS) {
return val_price;
}
target->price = new_price;
} else if (new_price != -1.0) {
return STATUS_ERR_INVALID_PRICE;
}

return STATUS_SUCCESS;
}

/* TASK 5: Xóa sản phẩm khỏi danh sách theo mã ID */
int delete_product(ProductList *list, const char *id) {
```

```
if (list == NULL || id == NULL) {
return STATUS_ERR_NULL_PTR;
}

for (size_t i = 0; i < list->count; ++i) {
if (strcmp(list->items[i].id, id) == 0) {
/* Dồn các phần tử phía sau lên 1 vị trí bằng memmove an toàn */
if (i < list->count - 1) {
memmove(&list->items[i], &list->items[i + 1], (list->count - i - 1) * sizeof(Product));
}
list->count--;
return STATUS_SUCCESS;
}
}

return STATUS_ERR_NOT_FOUND;
}

/* TASK 6: Tìm kiếm sản phẩm theo ID */
Product* find_product_by_id(const ProductList *list, const char *id) {
if (list == NULL || id == NULL || list->items == NULL) {
return NULL;
}
for (size_t i = 0; i < list->count; ++i) {
if (strcmp(list->items[i].id, id) == 0) {
return &list->items[i];
}
}
return NULL;
}

/* TASK 6: Tìm kiếm sản phẩm theo tên (tìm chuỗi con, không phân biệt hoa thường) */
int search_products_by_name(const ProductList *list, const char *keyword, Product *results, size_t max_results, size_t
*out_count) {
if (list == NULL || keyword == NULL || results == NULL || out_count == NULL) {
return STATUS_ERR_NULL_PTR;
}

*out_count = 0;
for (size_t i = 0; i < list->count && *out_count < max_results; ++i) {
if (contains_ignore_case(list->items[i].name, keyword)) {
results[*out_count] = list->items[i];
(*out_count)++;
}
}

return STATUS_SUCCESS;
}
```

2. Kết quả phân tích

Biểu thức: x + y
Có thể overflow int32? SAT
Ví dụ giá trị gây overflow: {'x': -1, 'y': -2147483648}